Kabadiwala Connect — Project Record

Smart India Hackathon 2026 · Problem statement SIH26229 · Ministry of Mines (Clean & Green Technology)

This document records everything the team built, step by step, in simple language: what the app does, how it works inside, the tools we used, the problems we hit and how we solved them. It is updated after every build phase.

Last updated: 24 October 2026 (after Phase 2)

1. The problem in one page

India produces a huge amount of electronic waste (old phones, TVs, wires, batteries, circuit boards). Most of it is collected by informal scrap collectors called **kabadiwalas**. They usually:

- sell to the nearest local dealer at a low price, because they don't know the fair rate;
- have no record of what they sold, to whom, or how much they were paid;
- sometimes break or burn the waste (for example burning cables for copper), which is dangerous for their health and the environment;
- often cannot read well, use cheap Android phones, and have weak or no internet.

Meanwhile, **authorised recyclers** (registered with the government to recycle e-waste safely) want more material but can't reach these collectors.

2. Our solution

Kabadiwala Connect is an Android app that works **without internet** and **without needing to read**. A collector can:

1. Take a photo of the scrap.
2. Pick what it is by tapping a picture (circuit board, copper wire, battery...).
3. Enter the weight with big + and − buttons.
4. **Hear** a fair price spoken aloud in their own language.
5. See nearby **authorised** recyclers, ranked by best rate, distance and pickup.
6. Hand over the scrap and get a **QR code** — a tamper-proof digital receipt.
7. The recycler scans the QR, checks it, and records the payment (cash or UPI).
8. The collector's **earnings ledger** updates. Everything syncs when internet comes back.

Who uses it

User	What they do in the app
Collector (kabadiwala)	Sells scrap, hears prices, finds recyclers, keeps an earnings record
Recycler	Scans the collector's QR, confirms weight and payment, sets their rates
Admin (our team)	Updates the price list and recycler list

Design rules we follow (for low-literacy users)

Rule	Why
No screen needs reading: every button = big icon + 1–2 words + sound	Many users can't read comfortably
At most 4 main choices per screen; buttons at least 56 dp tall	Easy to tap, not confusing
Prices are always spoken, never only shown	Numbers on screen are not enough
Colour is never the only signal: green + tick = done, amber + clock = waiting, red + warning = danger	Works for colour-blind users too

Rule	Why
Works offline after first login	Internet is patchy where collectors work
Cash is as important as UPI	Most collectors are paid in cash
All text comes from language files, never typed into the code	So we can add languages easily

3. Technology we chose (and why)

Part	Tool	Why we chose it (simple reason)
App	**Flutter** (Dart language)	One codebase, fast on cheap Android phones
State management	**Riverpod**	Keeps screens updated automatically when data changes
Database	**Cloud Firestore** (Firebase)	Works offline: saves on the phone first, uploads later
Login	**Firebase Phone Auth**	Collectors log in with their phone number + OTP, no passwords
Voice	**Android Text-to-Speech** + recorded clips	Speaks prices; recorded clips for languages Android can't speak
Tamper-proof receipt	**SHA-256 hash** + **QR code**	Any change to the record changes the code, so cheating is visible
Scripts	**Python** (pandas, firebase-admin)	Turns the team's spreadsheet data into app data
Cost	Firebase free "Spark" plan	No money needed for the prototype

Minimum phone: Android 7 (released 2016) or newer. The app works on all newer versions up to Android 16.

4. How we set up the computer (Mac)

Before writing any code, we installed the tools. Here is what each one is for:

Tool	What it does
Homebrew	An "app store" for developer tools on Mac
Java 17	Needed by Android's build system
Flutter 3.47	The framework the app is written in (includes Dart)
Android SDK 36 + build tools	Turns our code into an Android app (APK)
Android NDK	Compiles small native parts used by some plugins
Gradle 9.3	Android's build tool; downloads libraries and builds the APK
Firebase CLI	Lets us log in to Firebase and upload security rules from the terminal
FlutterFire CLI	Connects the Flutter app to our Firebase project
Python virtual environment	A private box for our Python scripts' libraries
Xcode Command Line Tools	Apple's compiler; `flutter test` needs it

The full step-by-step guide for teammates is in `docs/SETUP.md`.

Problems we hit during setup, and how we fixed them

Problem	Cause	Fix
Downloads were very slow (about 200 KB/s)	Slow internet connection	Ran downloads in the background and resumed them; switched to smaller downloads where possible
First Android build failed: "Operation timed out" while downloading Gradle	Our internet provider could not reach **one** of GitHub's 4 download servers (185.199.109.133). Java kept picking that one.	Downloaded Gradle with `curl` (which picked a working server), checked its SHA-256 checksum, and placed it where Gradle expects it
`flutter test` failed: "CarbonCore/Folders.h file not found"	The Mac's Command Line Tools installation was damaged (a file was missing)	Deleted and reinstalled the Command Line Tools
Android NDK download (900 MB) kept stalling	Slow internet; Android's downloader deletes the partial file if stopped	Downloaded it with `curl` in resume mode, verified the official SHA-1 checksum, then installed it manually
Wanted minimum Android 6 (API 23)	Current Flutter no longer supports Android 6	Set the minimum to Android 7 (API 24); almost no phones still run Android 6

Firebase setup (done in the browser by the team)

1. Created the Firebase project `kabadiwala-connect` (free Spark plan, no Google Analytics).
2. Created the Firestore database `(default)`, **Standard edition**, location **Mumbai (asia-south1)** — closest to our users, so faster.
3. Turned on **Phone sign-in** and added test phone numbers (fake numbers with a fixed OTP, so we can test without real SMS).
4. Logged in with `firebase login`.
5. Downloaded a **service-account key** (a secret file that lets our Python script write data). It is stored as `scripts/serviceAccount.json` and is **never** uploaded to git.
6. Ran `flutterfire configure`, which registered the Android app with Firebase and generated the connection settings (`firebase_options.dart`, `google-services.json`).

5. How the project is organised

Folder	What's inside
`app/`	The Flutter Android app
`app/lib/core/`	Shared building blocks: languages, audio, theme, hashing, location, sync
`app/lib/data/`	Data models, the code that reads/writes the database, offline seed data
`app/lib/features/`	One folder per screen group: onboarding, home, selling, prices, recyclers, handover, ledger, safety, recycler mode
`app/assets/`	Language files, sounds, icons, offline seed data
`app/test/`	Automatic tests that check the logic is correct

Folder	What's inside
`data/`	Spreadsheets (CSV) the team fills from the field study: materials, prices, recyclers, areas
`scripts/`	Python scripts that load the spreadsheets into the app and the database
`docs/`	Plan, setup guide, this record
`firestore.rules`	Security rules: who can read and write what in the database

6. What we built, phase by phase

The plan has 14 phases (0–13). Each phase ends with automatic tests, a code check (`flutter analyze`) and a saved version in git.

Phase 0 — Setup and scaffolding ✓

Goal: an empty app that runs, connects to Firebase, and has the right look.

What we made:

- Created the Flutter app (Android only, package `in.kabadiwalaconnect`).
- Added all the libraries the plan needs (camera, location, voice, QR, database...).
- Created the folder layout above.
- **Big-button theme:** every button at least 56 dp tall, large bold text, high-contrast colours, and three status colours (green, amber, red) that are always paired with an icon.
- A placeholder home screen that shows a **green tick** if Firebase connected or an **amber clock** if not. The app never crashes or blocks if Firebase is missing — it just keeps working.

Phase 1 — Data layer and seed data ✓

Goal: the app knows about materials, prices and recyclers, even with no internet on the very first run.

The data (spreadsheets in `data/`):

File	What it holds
`materials.csv`	8 scrap types: circuit board, copper wire, phone battery, vehicle battery, old TV, motor, mixed plastic, old phone. Each has an icon and a hazard level (none / low / high)
`prices.csv`	Price range per kg (low–high), last week's price, and the local dealer's price (for comparing earnings later)
`recyclers.csv`	Recycler name, phone, location, whether authorised, registration number, pickup or not, and their rate per material
`areas.csv`	A few areas with map coordinates, used if a collector doesn't allow location access

All rows are clearly marked **placeholder** for now. We will replace them with real data from our field study. We never invent real business names or phone numbers.

How the data gets into the app:

- `scripts/csv_to_assets.py` turns the spreadsheets into small files that are **built into the app** (`app/assets/seed/`). This is the offline backup.

- `scripts/seed_firestore.py` uploads the same data to the Firestore database.

Money and weight are stored as whole numbers: money in **paise** (Rs 1 = 100 paise) and weight in **grams**. Computers make small rounding errors with decimal numbers (for example 0.1 + 0.2 = 0.30000000000000004), which is unacceptable for money. Whole numbers never have this problem.

Offline-first reading: when a screen needs the price list, the app shows the built-in copy **immediately**, then switches to the live database copy as soon as it's available. So the app is never empty and never waits for internet.

Offline-first writing: when the collector saves something (a lot, a handover), Firestore saves it **on the phone instantly** and uploads it when internet returns. Every record gets a unique ID made on the phone (a UUID), so even if the same record is sent twice, it never becomes two records.

Data models (the "shape" of each record): Material, Price, Recycler, Area, User, Lot (one batch of scrap), Handover, Confirmation, Ledger entry.

One change from the original plan: the plan had the recycler update the collector's handover record. But if **both** phones are offline, the recycler can't update a record that hasn't reached the server yet. So the recycler writes a separate **Confirmation** record with the same ID. The collector's app joins them. This works even when both phones are offline.

Security rules (`firestore.rules`, uploaded to Firebase):

Data	Who can read	Who can write
Materials, prices, areas	Anyone logged in	Admin only
Recyclers	Anyone logged in	Admin; an invited recycler can link their login and edit only their own rates
A user's profile	Only that user	Only that user (cannot make themselves admin)
Lots, ledger	Only the collector who owns them	Only that collector
Handovers	The collector and the receiving recycler	Collector creates it; after that it can **never be changed**
Confirmations	The collector and the recycler	Only the recycler who received the scrap

Phase 2 — Languages and voice ✓

Goal: the app speaks and shows text in the user's language, and switching language is instant.

Languages:

Code	Language	How it's spoken
hi	Hindi (default)	Android text-to-speech
en	English (for judges)	Android text-to-speech
mr	Marathi	Android text-to-speech
gbm	Garhwali (local language, Uttarakhand)	Recorded voice clips (Android can't speak Garhwali). Until recorded, falls back to Hindi

Each language has one file, for example `assets/i18n/hi.json`. Text can have slots, e.g. "You will get about {amount} rupees" — the app fills in the number.

How the app decides how to speak (3 steps):

1. If a **recorded clip** exists for this sentence in this language, play it. Numbers are built from recorded number clips.
2. Otherwise, if the phone's **text-to-speech** knows the language, speak it.
3. Otherwise, speak it in **Hindi**.

Speaking numbers from recorded clips (NumberSpeaker): Hindi-family numbers are irregular (e.g. 21 = "ikkees", 45 = "paintalees"), so we record every number from 0 to 99 plus "hundred", "thousand", "lakh", "rupees", "about". The app breaks any amount into these pieces using the Indian system:

Amount	Clips played
Rs 340	about · 3 · hundred · 40 · rupees
Rs 1,340	about · 1 · thousand · 3 · hundred · 40 · rupees
Rs 1,50,000	about · 1 · lakh · 50 · thousand · rupees

Hidden audio-check screen: long-press the app title to open a list of every sentence with a play button, so the team can check each language.

`scripts/check_i18n.py`**:** checks that every language file has every sentence, and lists which recorded clips are still missing.

7. Testing

Every important piece of logic has automatic tests that run with `flutter test`. Current status: **34 tests, all passing**.

What is tested	Examples
Theme	Buttons are at least 56 dp
Language files	Every language has exactly the same sentences as Hindi; slots are filled correctly
Language switching	Tapping a language changes the text instantly and remembers the choice
Data models	Every record can be saved and read back unchanged; unknown values fall back safely
Seed data	Prices are whole paise and low ≤ high; missing recycler rates are skipped; every row is marked placeholder
Offline first run	With no database at all, the app still gets the material list
NumberSpeaker	0, 7, 40, 99, 100, 340, 1000, 1340, 99999, 1,50,000 and 1 crore
Voice fallback	Recorded clips win; a missing number clip doesn't play half a sentence; Marathi TTS; Hindi fallback

8. Data honesty

- All prices, recyclers and areas are **placeholders** until the field study is done, and they are marked that way in the data.
- We never invent real business names or phone numbers.

- The QR hash proves a record **was not changed after it was created**. It is not a legal signature, and we say so in the app.

9. Words used in this document

Word	Meaning
APK	The installable file of an Android app
Firestore	Google's online database that also works offline
Offline-first	The app works fully without internet and syncs later
Seed data	A built-in copy of the price/recycler list for the very first run
Hash (SHA-256)	A "fingerprint" of data: change one letter and the fingerprint changes completely
QR code	A square barcode a phone camera can read
TTS	Text-to-speech: the phone reads text aloud
Paise	1/100 of a rupee; we store money in paise to avoid rounding errors
UUID	A random unique ID made on the phone
Security rules	Server-side rules that decide who may read or change each record
Git commit	A saved snapshot of the code we can always go back to

10. What's next

Phase	What it adds
3	Language picker, phone login with OTP, 4-digit PIN for offline unlock, home screen with 4 big tiles
4	Selling flow: photo → category → weight → spoken price, all offline
5	Today's rates board with up/down trend arrows
6	Recycler matching and ranking (best rate + nearest + pickup)
7	Handover with tamper-proof QR; recycler mode to scan, verify and confirm payment
8	Earnings ledger: today/this week, money still due, "sold elsewhere"
9–13	Safety guide, sync robustness, polish, optional extras, demo mode